JAVA PROGRAMMING

Practice Program Unit 4

Submitted by:

Faustena S

(2548313)

2 MDS A

Faculty:

Dr. Heamenth K S

Submitted on 06 Feb. 26

Department of Computer Science and Statistics

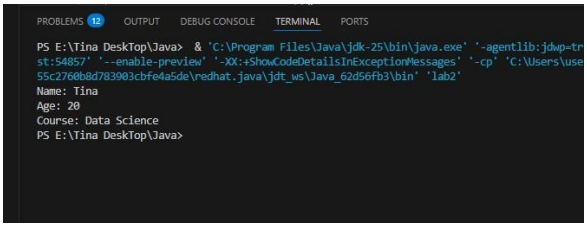
Christ (Deemed To Be University),

Yeswanthpur, Bangalore

Defining Classes and Objects

1. Student class with one

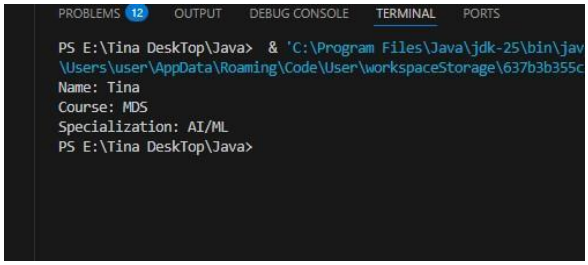
```
object class Student {     String
name;     int age;
   String course;
}

public class lab2 {     public static void
main(String[] args) {         Student s1 =
new Student();         s1.name = "Tina ";
s1.age = 20;         s1.course = "Data
Science";

       System.out.println("Name: " + s1.name);
       System.out.println("Age: " + s1.age);
       System.out.println("Course: " + s1.course);
   }
}
```



2. Two student objects with method

```
class Student {
   String name;
   String course;
}

class PostgraduateStudent extends Student {
```

```
    String specialization;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Specialization: " + specialization);
    }
}

public class p2 {     public static void
main(String[] args) {
        PostgraduateStudent pg = new PostgraduateStudent();
        pg.name = "Tina";
pg.course = "MDS";
pg.specialization = "AI/ML";

        pg.display();
    }
}
```



3. Department class with students

```
class Student1 {     String name;
int age;
    String course;
```

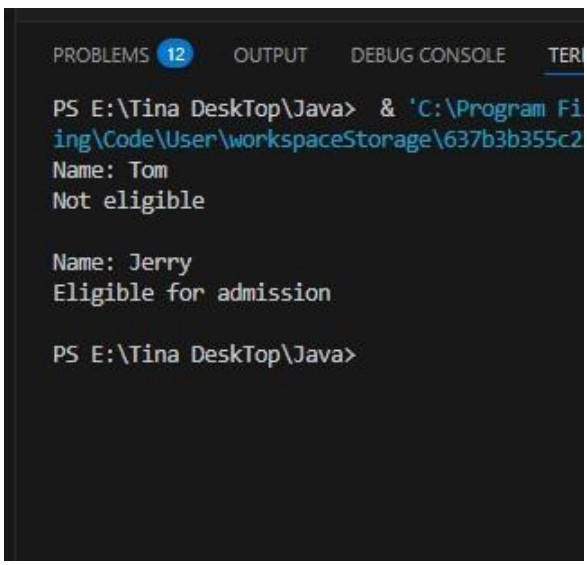
```
    void displayAdmission() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println();
    }
}

public class p3 {     public static void
main(String[] args) {         Student1 s1 =
new Student1();         s1.name = "Tina";
s1.age = 19;         s1.course = "Data
Science";

        Student1 s2 = new Student1();
s2.name = "Sruthi";         s2.age =
21;         s2.course = "Maths";

        s1.displayAdmission();
s2.displayAdmission();
    }
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\Program File
ing\Code\User\workspaceStorage\637b3b355c276
Name: Tina
Age: 19
Course: Data Science

Name: Sruthi
Age: 21
Course: Maths

PS E:\Tina DeskTop\Java>
```

4. Age eligibility check

```
class Student {

String name;     int

age;


   void checkEligibility() {

System.out.println("Name: " + name);         if(age

>= 18) {

            System.out.println("Eligible for admission");

        } else {

            System.out.println("Not eligible");

        }

        System.out.println();

    }

}


public class Test {     public static void

main(String[] args) {         Student s1 =
```

```
new Student();        s1.name = "Tom";
s1.age = 17;

        Student s2 = new Student();
s2.name = "Jerry";        s2.age =
19;

        s1.checkEligibility();
s2.checkEligibility();     }

}
```



5. Array of

students class

```
Student4 {     String
name;     int age;
boolean admitted;

    void display() {
```

```
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class p6 {     public static void
main(String[] args) {
        Student4[] students = new Student4[3];

        students[0] = new Student4();
students[0].name = "tina";
students[0].age = 20;
students[0].admitted = true;

        students[1] = new Student4();
students[1].name = "bolu";
students[1].age = 19;         students[1].admitted
= false;

        students[2] = new Student4();
students[2].name = "dolu";
students[2].age = 21;         students[2].admitted
= true;

        System.out.println("Admitted Students:");
for(int i = 0; i < students.length; i++) {
if(students[i].admitted) {
students[i].display();
```

```
        }
    }
  }
}
```

```
PS E:\Tina DeskTop\Java>
PS E:\Tina DeskTop\Java>  e:; cd 'e:\Tina Desk
cp' 'C:\Users\user\AppData\Roaming\Code\User\w
Admitted Students:
Name: tina, Age: 20
Name: dolu, Age: 21
PS E:\Tina DeskTop\Java>
```

Constructors and Methods 1.

Constructor with name and course

```
class Student5 {     String name;
  String course;

  Student5(String n, String c)
{        name = n;        course =
c;
  }

  void display() {
    System.out.println("Name: " + name);
    System.out.println("Course: " + course);
  }
}
```

```
public class p7 {     public static void
main(String[] args) {         Student5 s1 = new
Student5("Jo jo", "CSE");         s1.display();
    }
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\
iew' '-XX:+ShowCodeDetailsInExce
a_62d56fb3\bin' 'p7'
Name: Jo jo
Course: CSE
PS E:\Tina DeskTop\Java>
```

2. Auto-generate student

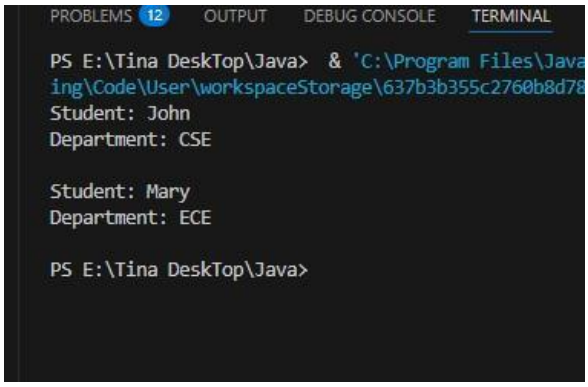
```
ID class Student6 {     String
name;     int studentId;
static int counter = 1000;

    Student6(String n) {
name = n;         studentId =
++counter;
    }

    void display() {
        System.out.println("ID: " + studentId + ", Name: " + name);
    }
}
```

```
public class p9 {     public static void
main(String[] args) {         Student6 s1 =
new Student6("tina");
        Student6 s2 = new Student6("terasia");
        Student6 s3 = new Student6("Carlin");

        s1.display();
s2.display();         s3.display();
    }
}
```

3. Overloaded constructors

```
class Student {
    String name;
String course;
boolean hostel;

    Student(String n, String c)
{         name = n;         course
= c;         hostel = false;
    }

    Student(String n, String c, boolean h)
{         name = n;         course = c;
hostel = h;
    }

    void display() {
```

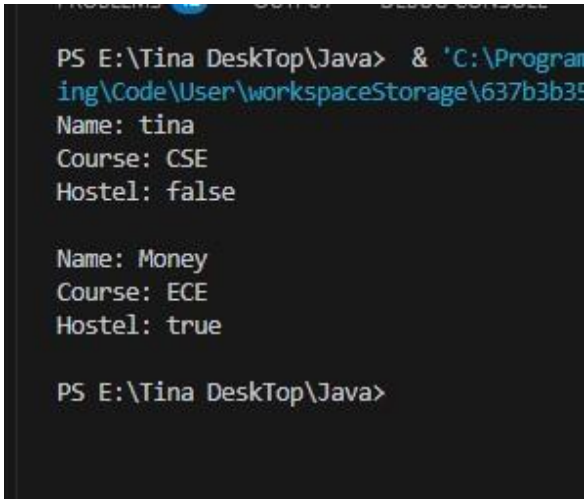
```
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);        System.out.println("Hostel: " + hostel);
        System.out.println();
    }
}

public class Test {     public static void
main(String[] args) {
        Student s1 = new Student("John", "CSE");
        Student s2 = new Student("Mary", "ECE", true);

        s1.display();
s2.display();
    }
}
```




```
class Student7 {
String name;
```

```
String course;
boolean hostel;

    Student7(String n, String c)
{        name = n;        course =
c;        hostel = false;
    }

    Student7(String n, String c, boolean h)
{        name = n;        course = c;
hostel = h;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Hostel: " + hostel);
        System.out.println();
    }
}

public class p10 {     public static void
main(String[] args) {        Student7 s1 =
new Student7("tina", "CSE");

        Student7 s2 = new Student7("Money", "ECE", true);
```

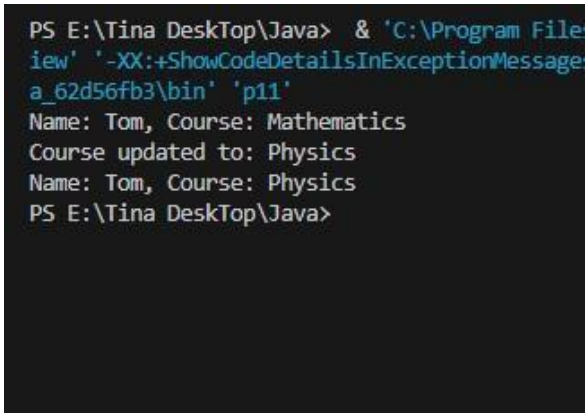
```
        s1.display();
s2.display();
    }
}
```



4. Methods to display and update

```
class Student8 {     String name;
    String course;

    Student8(String n, String c)
{         name = n;         course =
c;
    }

   void displayDetails() {
        System.out.println("Name: " + name + ", Course: " + course);
    }
```

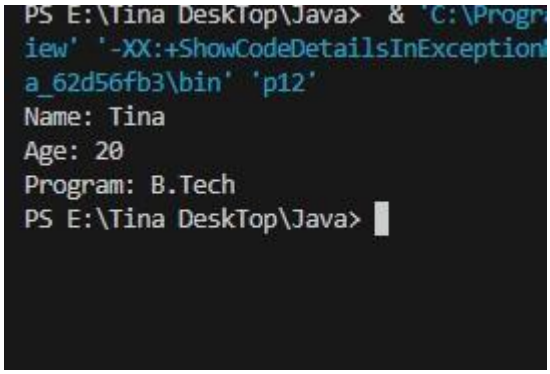
```
    void updateCourse(String newCourse) {
course = newCourse;
        System.out.println("Course updated to: " + course);
    }
}

public class p11 {     public static void
main(String[] args) {
        Student8 s1 = new Student8("Tom", "Mathematics");
s1.displayDetails();         s1.updateCourse("Physics");
s1.displayDetails();
    }
}
```



5. Constructor then method call

```
class Student9 {
String name;     int
age;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
```

```
    }
}

class UndergraduateStudent extends Student9 {
    String program;

    void showProgram() {
        System.out.println("Program: " + program);
    }
}

public class p12 {     public static void
main(String[] args) {
        UndergraduateStudent ug = new UndergraduateStudent();
ug.name = "Tina";         ug.age = 20;         ug.program =
"B.Tech";

        ug.display();
ug.showProgram();    }

}
```



Inheritance

1. Base and derived

```
class class Student {
    String name;
    int age;

    void display() {

System.out.println("Name
: " + name);

System.out.println("Age:
" + age);
    }
}

class
UndergraduateStudent
extends Student {
    String program;

    void showProgram() {

System.out.println("Progr
am: " + program);
    }
}
```

```
public class Test {
    public static void
main(String[] args) {

UndergraduateStudent ug
= new
UndergraduateStudent();
        ug.name = "Tina";
        ug.age = 20;
        ug.program =
"MDS";

        ug.display();
        ug.showProgram();
    }
}
```

```
        at java.base/java.lang.ClassLoader.loadClass
        ... 1 more
PS E:\Tina DeskTop\Java> javac Test.java
PS E:\Tina DeskTop\Java> java Test.java
Name: Tina
Age: 20
Program: MDS
PS E:\Tina DeskTop\Java>
```

2. PostgraduateStudent

with specialization

```
class Student {
    String name;
    String course;
}
```

```
class PostgraduateStudent extends Student {
    String specialization;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Specialization: " +
specialization);
    }
}

public class Test {
    public static void main(String[] args) {
        PostgraduateStudent pg = new
PostgraduateStudent();
        pg.name = "Tina";
        pg.course = "MDS";
        pg.specialization = "Data Science";

        pg.display();
    }
}
```

```
PS E:\Tina DeskTop\Java>  e:; cd 'e:\Tina DeskTop\Java'; & '
sInExceptionMessages' '-cp' 'C:\Users\user\AppData\Roaming\C
_62d56fb3\bin' 'Test'
Name: Tina
Course: MDS
Specialization: Data Science
PS E:\Tina DeskTop\Java>
```

Method Overriding

```java
class Student {
    String name;
    int age;
    String course;

    Student(String n, int a, String c) {
        name = n;
        age = a;
        course = c;
    }

    int calculateFee() {
        return 10000;
    }

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println("Fee: " + calculateFee());
    }
}

class UndergraduateStudent extends Student {
    String branch;

    UndergraduateStudent(String n, int a, String c, String b) {
        super(n, a, c);
```

```
        branch = b;
    }

    int calculateFee() {
        return 15000;
    }
}

class PostgraduateStudent extends Student {
    String specialization;

    PostgraduateStudent(String n, int a, String c, String s) {
        super(n, a, c);
        specialization = s;
    }

    int calculateFee() {
        return 20000;
    }
}

public class AdmissionSystem {

    static void processFee(Student s) {
        System.out.println("Processing fee for: " + s.name);
        System.out.println("Fee Amount: " + s.calculateFee());
    }

    public static void main(String[] args) {

        System.out.println("1. Student class with calculateFee()");
        Student s1 = new Student("Eleven", 20, "General");
        s1.displayDetails();

        System.out.println("\n2. Override in UndergraduateStudent");
        UndergraduateStudent ug1 = new UndergraduateStudent("Mike", 19,
"B.Tech", "CSE");
        ug1.displayDetails();

        System.out.println("\n3. Override in PostgraduateStudent");
```

```
        PostgraduateStudent pg1 = new PostgraduateStudent("Dustin", 23,
"M.Tech", "AI/ML");
        pg1.displayDetails();

        System.out.println("\n 4. Parent reference calling overridden
methods ");
        Student s2 = new UndergraduateStudent("Lucas", 20, "B.Tech",
"ECE");
        Student s3 = new PostgraduateStudent("Max", 24, "M.Tech", "Data
Science");

        System.out.println("Using parent reference for UG student:");
        s2.displayDetails();

        System.out.println("Using parent reference for PG student:");
        s3.displayDetails();

        System.out.println("\n 5. Runtime behavior - Admission Fee
Processing System");
        Student[] students = new Student[5];

        students[0] = new Student("Will", 21, "Diploma");
        students[1] = new UndergraduateStudent("Nancy", 19, "B.Tech",
"IT");
        students[2] = new PostgraduateStudent("Steve", 25, "M.Tech",
"Cybersecurity");
        students[3] = new UndergraduateStudent("Robin", 20, "B.Sc",
"Mathematics");
        students[4] = new PostgraduateStudent("Hopper", 24, "MBA",
"Finance");

        System.out.println("Processing fees for all admitted students:");
        for (int i = 0; i < students.length; i++) {
            processFee(students[i]);
        }

        System.out.println("\nTotal fees collected:");
        int totalFee = 0;
        for (int i = 0; i < students.length; i++) {
            totalFee += students[i].calculateFee();
```

```
        }
        System.out.println("Total: Rs. " + totalFee);
    }
}
```

```
1. Student class with calculateFee()
Name: Eleven
Age: 20
Course: General
Fee: 10000

2. Override in UndergraduateStudent
Name: Mike
Age: 19
Course: B.Tech
Fee: 15000

3. Override in PostgraduateStudent
Name: Dustin
Age: 23
Course: M.Tech
Fee: 20000

 4. Parent reference calling overridden methods
Using parent reference for UG student:
Name: Lucas
Age: 20
Course: B.Tech
Fee: 15000
Using parent reference for PG student:
Name: Max
Age: 24
Course: M.Tech
Fee: 20000
```

```
 5. Runtime behavior - Admission Fee Processing System
Processing fees for all admitted students:
Processing fee for: Will
Fee Amount: 10000
Processing fee for: Nancy
Fee Amount: 15000
Processing fee for: Steve
Fee Amount: 20000
Processing fee for: Robin
Fee Amount: 15000
Processing fee for: Hopper
Fee Amount: 20000

Total fees collected:
Total: Rs. 80000
```

Polymorphism

```java
class Student {
    String name;
    String course;

    Student(String name, String course) {
        this.name = name;
        this.course = course;
    }

    void admitStudent(String name) {
        System.out.println("Student admitted: " + name);
    }

    void admitStudent(String name, String course) {
        System.out.println("Student admitted: " + name + ", Course: " +
course);
    }

    void admitStudent(String name, String course, int age) {
        System.out.println("Student admitted: " + name + ", Course: " +
course + ", Age: " + age);
    }

    int calculateFee() {
        return 10000;
    }

    int calculateFee(String courseType) {
        if (courseType.equalsIgnoreCase("UG"))
            return 15000;
        else if (courseType.equalsIgnoreCase("PG"))
            return 20000;
        else
            return 10000;
    }
}

class UndergraduateStudent extends Student {

    UndergraduateStudent(String name, String course) {
```

```
        super(name, course);
    }

    int calculateFee() {
        return 15000;
    }
}

class PostgraduateStudent extends Student {

    PostgraduateStudent(String name, String course) {
        super(name, course);
    }

    int calculateFee() {
        return 20000;
    }
}

public class AdmissionSystem {

    static void processFee(Student s) {
        System.out.println("Processing fee for " + s.name);
        System.out.println("Fee Amount: " + s.calculateFee());
    }

    public static void main(String[] args) {

        Student baseStudent = new Student("Eleven", "General");
        baseStudent.admitStudent("Eleven");
        baseStudent.admitStudent("Eleven", "General");
        baseStudent.admitStudent("Eleven", "General", 20);

        System.out.println("Fee by course type UG: " +
baseStudent.calculateFee("UG"));
        System.out.println("Fee by course type PG: " +
baseStudent.calculateFee("PG"));

        Student s1 = new UndergraduateStudent("Mike", "B.Tech");
        Student s2 = new PostgraduateStudent("Dustin", "M.Tech");
```

```java
        System.out.println("\nRuntime Polymorphism:");
        processFee(s1);
        processFee(s2);

        Student[] students = new Student[4];
        students[0] = new Student("Will", "Diploma");
        students[1] = new UndergraduateStudent("Lucas", "B.Tech");
        students[2] = new PostgraduateStudent("Max", "M.Tech");
        students[3] = new UndergraduateStudent("Nancy", "B.Sc");

        System.out.println("\nDynamic Method Dispatch using Array:");
        for (Student s : students) {
            System.out.println(s.name + " Fee: " + s.calculateFee());
        }
    }
}
```

```
PS E:\Tina DeskTop\Java>  e:; cd 'e:\Tina DeskTop\Java';
cp' 'C:\Users\user\AppData\Roaming\Code\User\workspaceSto
Student admitted: Eleven
Student admitted: Eleven, Course: General
Student admitted: Eleven, Course: General, Age: 20
Fee by course type UG: 15000
Fee by course type PG: 20000

Runtime Polymorphism:
Processing fee for Mike
Fee Amount: 15000
Processing fee for Dustin
Fee Amount: 20000

Dynamic Method Dispatch using Array:
Will Fee: 10000
Lucas Fee: 15000
Max Fee: 20000
Nancy Fee: 15000
PS E:\Tina DeskTop\Java>
```

\

Encapsulation and Abstraction

```java
interface ScholarshipEligible {
```

```java
    boolean isEligibleForScholarship();
}

abstract class StudentAdmission {
    abstract int calculateFee();
}

class Student extends StudentAdmission implements ScholarshipEligible {

    private String name;
    private int age;
    private String course;
    private final int admissionNumber;

    Student(String name, int age, String course, int admissionNumber) {
        this.name = name;
        this.admissionNumber = admissionNumber;
        setAge(age);
        this.course = course;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age >= 17) {
            this.age = age;
        } else {
            System.out.println("Invalid age. Minimum age is 17.");
        }
    }
```

```
    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    public int getAdmissionNumber() {
        return admissionNumber;
    }

    int calculateFee() {
        return 10000;
    }

    public boolean isEligibleForScholarship() {
        return age < 22;
    }

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println("Admission No: " + admissionNumber);
        System.out.println("Fee: " + calculateFee());
        System.out.println("Scholarship Eligible: " +
isEligibleForScholarship());
    }
}

class UndergraduateStudent extends Student {

    UndergraduateStudent(String name, int age, String course, int
admissionNumber) {
        super(name, age, course, admissionNumber);
    }

    int calculateFee() {
```

```
        return 15000;
    }
}

class PostgraduateStudent extends Student {

    PostgraduateStudent(String name, int age, String course, int
admissionNumber) {
        super(name, age, course, admissionNumber);
    }

    int calculateFee() {
        return 20000;
    }
}

public class AdmissionSystem {
    public static void main(String[] args) {

        Student s1 = new UndergraduateStudent("Eleven", 19, "B.Tech",
101);
        Student s2 = new PostgraduateStudent("Mike", 23, "M.Tech", 102);

        s1.displayDetails();
        System.out.println();
        s2.displayDetails();
    }
}
```

```
bfe4a5de\redhat.java\jdt_ws\Java_62d56fb3\bin' 'A
Name: Eleven
Age: 19
Course: B.Tech
Admission No: 101
Fee: 15000
Scholarship Eligible: true

Name: Mike
Age: 23
Course: M.Tech
Admission No: 102
Fee: 20000
Scholarship Eligible: false
PS E:\Tina DeskTop\Java> ^C
PS E:\Tina DeskTop\Java>
PS E:\Tina DeskTop\Java>  e:; cd 'e:\Tina DeskTop
localhost:61649' '--enable-preview' '-XX:+ShowCod
```

Packages in Java